4. Task 3: Vulnerability Analysis

4.1 Crash Triage Methodology

After fuzzing each program with ASAN enabled as described above, a large amount of crashes had been generated and saved for each of the target programs.

Initially, crashes were separated based on the sig value in their file name, which denotes what type of crash it was. From there, the vulnerable programs were run in GDB with the different crashes being provided as input. Through GDB, it was possible to view more data about the crash, such as where exactly in memory the error had occurred or via backtrace, the path through the program the fuzzer had taken.

After analysing crashes in GDB, the team was able to group them by type of crash, where in the program the crash had occurred and exactly what had happened. For example, multiple crashes that are all stack buffer overflows that all occur on the same variable in the same place in memory can be safely grouped as one vulnerability.

4.2 Discovered Vulnerabilities

4.2.1 Vulnerability 1

Field	Details
Vulnerability Name	`sprintf() vulnerability in process_telemetry_directive()`
CWE Classification	`CWE-121: Stack-based buffer overflow`
Target Component	`sentinel_telemetry.c`

Vulnerable Location	`in process_telemetry_directive(), line 100`
	
	`sprintf(status_output, "Directive parsed: %s=%s", formatted_key, formatted_value);`

Reproducing Input File	`crashing_inputs/telemetry_crash_01.conf`

Triggering Input & Reproduction Command:

```
# Provide command to reproduce the crash
subsystem_mode=nominal
sensor_id_primary=101
cal_factor_thermal=1.05
log_event=subsystem_initialised
streammmmmmmmmmmmmmmmmmmmmmmmmmmmmmmmmmmmm_multiplier=4
```

Root Cause Analysis:

This vulnerability exists because the program `sentinel_telemetry.c` allocates 64 characters to the string for displaying each received telemetry signal, `status_output`. If a received telemetry signal's key and value combine to be over 64 characters, the `sprintf()` function will overflow `status_output`, causing a stack based buffer overflow.

GDB / Sanitiser Evidence:

```
Address 0xfbfff5c0e960 is located in stack of thread T0 at offset 352 in frame
    #0 0xaaaaaabbedf8  (/home/amber.guest/assignment2/sentinel_telemetry+0x11edf8) (BuildId: e14fb6ffb5bd4e386c2301ce21eb
b441116f5815c)

  This frame has 3 object(s):
    [32, 96) 'formatted_key' (line 65)
    [128, 256) 'formatted_value' (line 66)
    [288, 352) 'status_output' (line 67) <== Memory access at offset 352 overflows this variable
HINT: this may be a false positive if your program uses some custom stack unwind mechanism, swapcontext or vfork
      (longjmp and C++ exceptions *are* supported)
SUMMARY: AddressSanitizer: stack-buffer-overflow (/home/amber.guest/assignment2/sentinel_telemetry+0x5f244) (BuildId: e14
4fb6ffb5bd4e386c2301ce21eb441116f5815c)
Shadow bytes around the buggy address:
  0xfbfff5c0e680: f5 f5 f5 f5 f5 f5 f5 f5 f5 f5 f5 f5 f5 f5 f5 f5
  0xfbfff5c0e700: f5 f5 f5 f5 f5 f5 f5 f5 f5 f5 f5 f5 f5 f5 f5 f5
  0xfbfff5c0e780: f5 f5 f5 f5 f5 f5 f5 f5 f5 f5 f5 f5 f5 f5 f5 f5
  0xfbfff5c0e800: f1 f1 f1 f1 00 00 00 00 00 00 00 00 f2 f2 f2 f2
  0xfbfff5c0e880: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
=>0xfbfff5c0e900: f2 f2 f2 f2 00 00 00 00 00 00 00 00[f3]f3 f3 f3
  0xfbfff5c0e980: f3 f3 f3 f3 f3 f3 f3 f3 f3 f3 f3 f3 f3 f3 f3 f3
  0xfbfff5c0ea00: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0xfbfff5c0ea80: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0xfbfff5c0eb00: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0xfbfff5c0eb80: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
Shadow byte legend (one shadow byte represents 8 application bytes):
  Addressable:           00
  Partially addressable: 01 02 03 04 05 06 07
  Heap left redzone:       fa
  Freed heap region:       fd
  Stack left redzone:      f1
  Stack mid redzone:       f2
  Stack right redzone:     f3
  Stack after return:      f5
  Stack use after scope:   f8
  Global redzone:          f9
  Global init order:       f6
  Poisoned by user:        f7
  Container overflow:      fc
  Array cookie:            ac
  Intra object redzone:    bb
  ASan internal:           fe
  Left alloca redzone:     ca
  Right alloca redzone:    cb
==1964879==ABORTING
```

Exploitability Assessment:

As a stack buffer overflow vulnerability, this could be a severe exploit, allowing attacks to write arbitrary data to the stack or a return address. This could potentially allow attacks to jump the code to any function they want by manipulating the return address.

4.2.2 Vulnerability 2

Field	Details
Vulnerability Name	`printf() printing unsanitised input, attempts to print a special format specifier`
CWE Classification	`CWE-134: Use of Externally-Controlled Format String`
Target Component	`sentinel_telemetry.c`

Vulnerable Location	`in log_telemetry_event(), on line 43`
	
	`printf(event_message);`

Reproducing Input File	`crashing_files/telemetry_crash_02.conf`

Triggering Input & Reproduction Command:

```
# Provide command to reproduce the crash
subsystr=20
%n%n%x%
```

Root Cause Analysis:

This vulnerability exists because the program `sentinel_telemetry.c` prints unsanitised input on line 43, with the function `printf(event_message);`. This means that an attack can have special format strings such as `%x` or `%n` that try to access parameters that weren't provided in the `printf()` function. This allows attacks to access other values in the stack since `printf()` will look at the stack for values to print if none are provided.

GDB / Sanitiser Evidence:

```
==2322==AddressSanitizer: WARNING: unexpected format specifier in printf interceptor: %
 (reported once per process)
AddressSanitizer:DEADLYSIGNAL
=================================================================
==2322==ERROR: AddressSanitizer: SEGV on unknown address 0x000000000000 (pc 0xf70c306f9104 bp 0xffffcd160480 sp 0xffffcd15ff20 T0)
==2322==The signal is caused by a WRITE memory access.
==2322==Hint: address points to the zero page.
==2322==WARNING: invalid path to external symbolizer!
==2322==WARNING: Failed to use and restart external symbolizer!
    #0 0xf70c306f9104  (/usr/lib/aarch64-linux-gnu/libc.so.6+0x59104) (BuildId: 5697d481680bea92debb50f8a02893668fb7f6e1)
    #1 0xf70c306fa044  (/usr/lib/aarch64-linux-gnu/libc.so.6+0x5a044) (BuildId: 5697d481680bea92debb50f8a02893668fb7f6e1)
    #2 0xf70c30711170  (/usr/lib/aarch64-linux-gnu/libc.so.6+0x71170) (BuildId: 5697d481680bea92debb50f8a02893668fb7f6e1)
    #3 0xaee910350238  (/home/amber.guest/assignment2/sentinel_telemetry+0x60238) (BuildId: f1a4596859e650c4e4c9f544232821970bf2aef9)
    #4 0xaee91040ec8c  (/home/amber.guest/assignment2/sentinel_telemetry+0x11ec8c) (BuildId: f1a4596859e650c4e4c9f544232821970bf2aef9)
    #5 0xaee91040fee8  (/home/amber.guest/assignment2/sentinel_telemetry+0x11fee8) (BuildId: f1a4596859e650c4e4c9f544232821970bf2aef9)
    #6 0xaee910410728  (/home/amber.guest/assignment2/sentinel_telemetry+0x120728) (BuildId: f1a4596859e650c4e4c9f544232821970bf2aef9)
    #7 0xaee910410af4  (/home/amber.guest/assignment2/sentinel_telemetry+0x120af4) (BuildId: f1a4596859e650c4e4c9f544232821970bf2aef9)
    #8 0xf70c306c2f18  (/usr/lib/aarch64-linux-gnu/libc.so.6+0x22f18) (BuildId: 5697d481680bea92debb50f8a02893668fb7f6e1)
    #9 0xf70c306c3058  (/usr/lib/aarch64-linux-gnu/libc.so.6+0x23058) (BuildId: 5697d481680bea92debb50f8a02893668fb7f6e1)
    #10 0xaee9103272ac  (/home/amber.guest/assignment2/sentinel_telemetry+0x372ac) (BuildId: f1a4596859e650c4e4c9f544232821970bf2aef9)

==2322==Register values:
 x0 = 0x0000000000000037   x1 = 0x0000000000000037   x2 = 0x0000000000000000   x3 = 0x0003cc30b9c01080
 x4 = 0x000000000000006e   x5 = 0x0000000000000000   x6 = 0x0000000000000000   x7 = 0x0000000000000000
 x8 = 0x0000000000000000   x9 = 0x00000000ffffffff  x10 = 0x0000f70c3011e075  x11 = 0x000000000000000a
x12 = 0x0000f30c2e70045e  x13 = 0x0000000000000000  x14 = 0x0000000000000073  x15 = 0x0000000000000000
x16 = 0x0000000000000001  x17 = 0x0000000000000000  x18 = 0x0000000000002000  x19 = 0x0000ffffcd160528
x20 = 0x0000ffffcd1604e0  x21 = 0x0000000000000000  x22 = 0x0000ffffcd1607f0  x23 = 0x0000f70c306f7b7c
x24 = 0x0000000000000000  x25 = 0x0000000000000000  x26 = 0x0000f30c2e700458  x27 = 0x0000000000000000
x28 = 0x0000000000000000   fp = 0x0000ffffcd160480   lr = 0x0000f70c306f81d8   sp = 0x0000ffffcd15ff20
AddressSanitizer can not provide additional info.
SUMMARY: AddressSanitizer: SEGV (/usr/lib/aarch64-linux-gnu/libc.so.6+0x59104) (BuildId: 5697d481680bea92debb50f8a02893668fb7f6e1)
==2322==ABORTING
```

Exploitability Assessment:

An attack could potentially exploit the unsanitised printing of special format parameters to reveal values on the stack that an attacker shouldn't have access to. Using the special format parameter `%n`, attackers may also be able to write values to the stack.

4.2.3 Vulnerability 3

Field	Details
Vulnerability Name	`strcpy() error in process_frame_label()`
CWE Classification	`CWE-121: Stack-based Buffer Overflow`
Target Component	`sentinel_payload.c`

Vulnerable Location	`in process_frame_label(), on line 29`
	
	`strcpy(local_label, label);`

Reproducing Input File	`crashing_inputs/payload_crash_01.bin`

Triggering Input & Reproduction Command:

```
payload_data_1 = b"PAYLOAD_FRAME 8 8 1 RADAR_SCAN_01"
payload_data_2 =
b"ioubhduoychbwiuphefuoyberwouihfnipuwhebouifdbwipuefhipweuhfoiuwehbouif
hbuob"
```

```
with open("payload_crash_01.bin", "wb") as f:
    f.write(payload_data_1 + payload_data_2)
```

Root Cause Analysis:

In the function `process_frame_label()`, the unsafe command `strcpy();` is used on line 29. This command copies a string that has a max length of 128 chars (`char label_buffer[128]` on line 90) to a string that has a max length of 64 chars (`char local_label[MAX_LABEL_LENGTH]` on line 28). If the input payload is longer than 64 characters, this will cause a stack buffer overflow where the code writes beyond the string `local_label`.

GDB / Sanitiser Evidence:

```
Address 0xfbfff5a00060 is located in stack of thread T0 at offset 96 in frame
    #0 0xaaaaaabbe954  (/home/amber.guest/assignment2/sentinel_payload+0x11e954) (BuildId: e3d75cdf97
762494dd559b8239b5f03720a56080)

  This frame has 1 object(s):
    [32, 96) 'local_label' (line 28) <== Memory access at offset 96 overflows this variable
HINT: this may be a false positive if your program uses some custom stack unwind mechanism, swapconte
xt or vfork
      (longjmp and C++ exceptions *are* supported)
SUMMARY: AddressSanitizer: stack-buffer-overflow (/home/amber.guest/assignment2/sentinel_payload+0xc2
d20) (BuildId: e3d75cdf97762494dd559b8239b5f03720a56080)
Shadow bytes around the buggy address:
  0xfbfff59ffd80: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0xfbfff59ffe00: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0xfbfff59ffe80: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0xfbfff59fff00: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0xfbfff59fff80: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
=>0xfbfff5a00000: f1 f1 f1 f1 00 00 00 00 00 00 00 00[f3]f3 f3 f3
  0xfbfff5a00080: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0xfbfff5a00100: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0xfbfff5a00180: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0xfbfff5a00200: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0xfbfff5a00280: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
Shadow byte legend (one shadow byte represents 8 application bytes):
  Addressable:           00
  Partially addressable: 01 02 03 04 05 06 07
  Heap left redzone:       fa
  Freed heap region:       fd
  Stack left redzone:      f1
  Stack mid redzone:       f2
  Stack right redzone:     f3
  Stack after return:      f5
  Stack use after scope:   f8
  Global redzone:          f9
  Global init order:       f6
  Poisoned by user:        f7
  Container overflow:      fc
  Array cookie:            ac
  Intra object redzone:    bb
  ASan internal:           fe
  Left alloca redzone:     ca
  Right alloca redzone:    cb
==2155==ABORTING
[Inferior 1 (process 2155) exited with code 01]
```

Exploitability Assessment:

As another stack buffer overflow, this could allow an attacker to overflow memory on the stack,

causing crashes, or allowing for altering return addresses. Altering return addresses could potentially jump the program to any point.

4.2.4 Vulnerability 4

Field	Details
Vulnerability Name	`Heap Buffer Overflow in process_payload_data()`
CWE Classification	`CWE-122: Heap-based buffer overflow`
Target Component	`sentinel_payload.c`

Vulnerable Location	`In process_payload_data(), on line 84`
	
	`memcpy(global_frame→data, input_bytes, input_size);`

Reproducing Input File	`crashing_inputs/payload_crash_02.bin`

Triggering Input & Reproduction Command:

```
# Provide command to reproduce the crash
payload_data_1 = b"PAYLOAD_FRAME 8 8 24 RADAR_SCAN_01\n"
payload_data_2 =
b"ppiuqehiurfnhreiuvhripehrupifvuehipfrvhnewpiurhnvipurehnpviuheipurfvhi
preuhsfipvhrnepiuvnpirefhnpirehsipvhpirrhpiwhvpiuhrepiwhniprehipuehfiphf
piuehipsvherwipfhnweipunfipuenfiupewbnripuehpiufhnqepiufnipquebnrfipubnq
epirfhunpqiufhripquehnfipeuhnfipueqhfipueqhfipurqhepifuhreipqfunhvipunef
iphqrfipuefhripquehfipuqefhipuehfipuqehfipuhpiuqehiurfnhreiuvhripehrupif
vuehipfrvhnewpiurhnvipurehnpviuheipurfvhipreuhsfipvhrnepiuvnpirefhnpireh
sipvhpirrhpiwhvpiuhrepiwhniprehipuehfiphfpiuehipsvherwipfhnweipunfipuenf
iupewbnripuehpiufhnqepiufnipquebnrfipubnqepirfhunpqiufhripquehnfipeuhnfi
pueqhfipueqhfipurqhepifuhreipqfunhvipunefiphqrfipuefhripquehfipuqefhipue
hfipuqehfipuhpiuqehiurfnhreiuvhripehrupifvuehipfrvhnewpiurhnvipurehnpviu
heipurfvhipreuhsfipvhrnepiuvnpirefhnpirehsipvhpirrhpiwhvpiuhrepiwhnipreh
ipuehfiphfpiuehipsvherwipfhnweipunfipuenfiupewbnripuehpiufhnqepiufnipque
bnrfipubnqepirfhunpqiufhripquehnfipeuhnfipueqhfipueqhfipurqhepifuhreipqf
unhvipunefiphqrfipuefhripquehfipuqefhipuehfipuqehfipuhpiuqehiurfnhreiuvh
ripehrupifvuehipfrvhnewpiurhnvipurehnpviuheipurfvhipreuhsfipvhrnepiuvnpi
refhnpirehsipvhpirrhpiwhvpiuhrepiwhniprehipuehfiphfpiuehipsvherwipfhnwei
punfipuenfiupewbnripuehpiufhnqepiufnipquebnrfipubnqepirfhunpqiufhripqueh
nfipeuhnfipueqhfipueqhfipurqhepifuhreipqfunhvipunefiphqrfipuefhripquehfi
```

```
puqefhipuehfipuqehfipuhpiuqehiurfnhreiuvhripehrupifvuehipfrvhnewpiurhnvi
purehnpviuheipurfvhipreuhsfipvhrnepiuvnpirefhnpirehsipvhpirrhpiwhvpiuhre
piwhniprehipuehfiphfpiuehipsvherwipfhnweipunfipuenfiupewbnripuehpiufhnqe
piufnipquebnrfipubnqepirfhunpqiufhripquehnfipeuhnfipueqhfipueqhfipurqhep
ifuhreipqfunhvipunefiphqrfipuefhripquehfipuqefhipuehfipuqehfipuhpiuqehiu
rfnhreiuvhripehrupifvuehipfrvhnewpiurhnvipurehnpviuheipurfvhipreuhsfipvh
rnepiuvnpirefhnpirehsipvhpirrhpiwhvpiuhrepiwhniprehipuehfiphfpiuehipsvhe
rwipfhnweipunfipuenfiupewbnripuehpiufhnqepiufnipquebnrfipubnqepirfhunpqi
ufhripquehnfipeuhnfipueqhfipueqhfipurqhepifuhreipqfunhvipunefiphqrfipuef
hripquehfipuqefhipuehfipuqehfipuhpiuqehiurfnhreiuvhripehrupifvuehipfrvhn
ewpiurhnvipurehnpviuheipurfvhipreuhsfipvhrnepiuvnpirefhnpirehsipvhpirrhp
iwhvpiuhrepiwhniprehipuehfiphfpiuehipsvherwipfhnweipunfipuenfiupewbnripu
ehpiufhnqepiufnipquebnrfipubnqepirfhunpqiufhripquehnfipeuhnfipueqhfipueq
hfipurqhepifuhreipqfunhvipunefiphqrfipuefhripquehfipuqefhipuehfipuqehfip
uhpiuqehiurfnhreiuvhripehrupifvuehipfrvhnewpiurhnvipurehnpviuheipurfvhip
reuhsfipvhrnepiuvnpirefhnpirehsipvhpirrhpiwhvpiuhrepiwhniprehipuehfiphfp
iuehipsvherwipfhnweipunfipuenfiupewbnripuehpiufhnqepiufnipquebnrfipubnqe
pirfhunpqiufhripquehnfipeuhnfipueqhfipueqhfipurqhepifuhreipqfunhvipunefi
phqrfipuefhripquehfipuqefhipuehfipuqehfipuhpiuqehiurfnhreiuvhripehrupifv
uehipfrvhnewpiurhnvipurehnpviuheipurfvhipreuhsfipvhrnepiuvnpirefhnpirehs
ipvhpirrhpiwhvpiuhrepiwhniprehipuehfiphfpiuehipsvherwipfhnweipunfipuenfi
upewbnripuehpiufhnqepiufnipquebnrfipubnqepirfhunpqiufhripquehnfipeuhnfip
ueqhfipueqhfipurqhepifuhreipqfunhvipunefiphqrfipuefhripquehfipuqefhipueh
fipuqehfipuhpiuqehiurfnhreiuvhripehrupifvuehipfrvhnewpiurhnvipurehnpviuh
eipurfvhipreuhsfipvhrnepiuvnpirefhnpirehsipvhpirrhpiwhvpiuhrepiwhniprehi
puehfiphfpiuehipsvherwipfhnweipunfipuenfiupewbnripuehpiufhnqepiufnipqueb
nrfipubnqepirfhunpqiufhripquehnfipeuhnfipueqhfipueqhfipurqhepifuhreipqfu
nhvipunefiphqrfipuefhripquehfipuqefhipuehfipuqehfipuhiuqehiurfnhreiuvhri
pehrupifvuehipfrvhnewpiurhnvipurehnpviuheipurfvhipreuhsfipvhrnepiuvnpire
fhnpirehsipvhpirrhpiwhvpiuhrepiwhniprehipuehfiphfpiuehipsvherwipfhnweipu
nfipuenfiupewbnripuehpiufhnqepiufnipquebnrfipubnqepirfhunpqiufhripquehnf
ipeuhnfipueqhfipueqhfipurqhepifuhreipqfunhvipunefiphqrfipuefhripquehfipu
qefhipuehfipuqehfipuh"

with open("payload_crash_02.bin", "wb") as f:
    f.write(payload_data_1 + payload_data_2)
```

Root Cause Analysis:

In the function `process_payload_data()`, the global frame is only allocated the memory for the `sizeof(payload_frame_t)`, but the `parse_payload_file()` function that passes the payload data will allocate and read as much payload data as is specified in the input's header. If the `stream_data` coming from `parse_payload_file` is larger than the size of the struct `payload_frame_t` there will be a heap buffer overflow error causing the program to crash.

GDB / Sanitiser Evidence:

```
==2985==ERROR: AddressSanitizer: heap-buffer-overflow on address 0xfd8ff6de0980 at pc 0xaaaaaab7a764 bp 0xffffffffedc0 sp 0xffffffffe5b0
WRITE of size 7578 at 0xfd8ff6de0980 thread T0
==2985==WARNING: invalid path to external symbolizer!
==2985==WARNING: Failed to use and restart external symbolizer!
    #0 0xaaaaaab7a760  (/home/amber.guest/assignment2/sentinel_payload+0xda760) (BuildId: e3d75cdf97762494dd559b8239b5f03720a56080)
    #1 0xaaaaaabbf354  (/home/amber.guest/assignment2/sentinel_payload+0x11f354) (BuildId: e3d75cdf97762494dd559b8239b5f03720a56080)
    #2 0xaaaaaabbf7e8  (/home/amber.guest/assignment2/sentinel_payload+0x11f7e8) (BuildId: e3d75cdf97762494dd559b8239b5f03720a56080)
    #3 0xaaaaaabbfb6c  (/home/amber.guest/assignment2/sentinel_payload+0x11fb6c) (BuildId: e3d75cdf97762494dd559b8239b5f03720a56080)
    #4 0xffffff7cd2f18  (/usr/lib/aarch64-linux-gnu/libc.so.6+0x22f18) (BuildId: 5697d481680bea92debb50f8a02893668fb7f6e1)
    #5 0xffffff7cd3058  (/usr/lib/aarch64-linux-gnu/libc.so.6+0x23058) (BuildId: 5697d481680bea92debb50f8a02893668fb7f6e1)
    #6 0xaaaaaaad706c  (/home/amber.guest/assignment2/sentinel_payload+0x3706c) (BuildId: e3d75cdf97762494dd559b8239b5f03720a56080)

0xfd8ff6de0980 is located 0 bytes after 1024-byte region [0xfd8ff6de0580,0xfd8ff6de0980)
allocated by thread T0 here:
    #0 0xaaaaaab7cbac  (/home/amber.guest/assignment2/sentinel_payload+0xdcbac) (BuildId: e3d75cdf97762494dd559b8239b5f03720a56080)
    #1 0xaaaaaabbf19c  (/home/amber.guest/assignment2/sentinel_payload+0x11f19c) (BuildId: e3d75cdf97762494dd559b8239b5f03720a56080)
    #2 0xaaaaaabbf7e8  (/home/amber.guest/assignment2/sentinel_payload+0x11f7e8) (BuildId: e3d75cdf97762494dd559b8239b5f03720a56080)
    #3 0xaaaaaabbfb6c  (/home/amber.guest/assignment2/sentinel_payload+0x11fb6c) (BuildId: e3d75cdf97762494dd559b8239b5f03720a56080)
    #4 0xffffff7cd2f18  (/usr/lib/aarch64-linux-gnu/libc.so.6+0x22f18) (BuildId: 5697d481680bea92debb50f8a02893668fb7f6e1)
    #5 0xffffff7cd3058  (/usr/lib/aarch64-linux-gnu/libc.so.6+0x23058) (BuildId: 5697d481680bea92debb50f8a02893668fb7f6e1)
    #6 0xaaaaaaad706c  (/home/amber.guest/assignment2/sentinel_payload+0x3706c) (BuildId: e3d75cdf97762494dd559b8239b5f03720a56080)

SUMMARY: AddressSanitizer: heap-buffer-overflow (/home/amber.guest/assignment2/sentinel_payload+0xda760) (BuildId: e3d75cdf97762494dd559b8239b
b5f03720a56080)
Shadow bytes around the buggy address:
  0xfd8ff6de0700: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0xfd8ff6de0780: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0xfd8ff6de0800: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0xfd8ff6de0880: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0xfd8ff6de0900: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
=>0xfd8ff6de0980:[fa]fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
  0xfd8ff6de0a00: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
  0xfd8ff6de0a80: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
  0xfd8ff6de0b00: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
  0xfd8ff6de0b80: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
  0xfd8ff6de0c00: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
Shadow byte legend (one shadow byte represents 8 application bytes):
  Addressable:           00
  Partially addressable: 01 02 03 04 05 06 07
  Heap left redzone:       fa
  Freed heap region:       fd
  Stack left redzone:      f1
  Stack mid redzone:       f2
  Stack right redzone:     f3
  Stack after return:      f5
  Stack use after scope:   f8
  Global redzone:          f9
  Global init order:       f6
  Poisoned by user:        f7
  Container overflow:      fc
  Array cookie:            ac
  Intra object redzone:    bb
  ASan internal:           fe
  Left alloca redzone:     ca
  Right alloca redzone:    cb
==2985==ABORTING
```

Exploitability Assessment:

A heap buffer overflow exploit can allow attackers to crash the program or overwrite variables they shouldn't have access to that are stored on the heap. Sophisticated heap overflow attacks could overwrite huge amounts of data that the program is relying on to run properly.

4.2.5 Vulnerability 5

Field	Details
Vulnerability Name	`Printing unsanitised special format parameters in log_payload_error()`
CWE Classification	`CWE-134: Use of Externally-Controlled Format String`
Target Component	`sentinel_payload.c`

Field	Details

Vulnerable Location	`In log_payload_error(), on line 35`
	
	`fprintf(stderr, error_description);`

Reproducing Input File	`crashing_inputs/payload_crash_03.bin`

Triggering Input & Reproduction Command:

```
# Provide command to reproduce the crash
%x%x%n
```

Root Cause Analysis:
If scanning for the expected header format doesn't work on line 98, line 99 directly passes the variable `header_line` to `log_payload_error()` which prints `header_line` to stderr without sanitising it, allowing for special format characters to be read. This can cause segmentation faults and reveal memory on the stack that shouldn't be accessible by the user.

GDB / Sanitiser Evidence:

```
(gdb) run crashing_inputs/payload_crash_03.bin
Starting program: /home/amber.guest/assignment2/sentinel_payload crashing_inputs/payload_crash_03.bin
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/aarch64-linux-gnu/libthread_db.so.1".
=== Sentinel-1 Scientific Payload Ingestion Subsystem ===
[PAYLOAD ERROR]
Program received signal SIGSEGV, Segmentation fault.
0x0000ffffff7d09104 in ?? () from /usr/lib/aarch64-linux-gnu/libc.so.6
(gdb) bt
#0  0x0000ffffff7d09104 in ?? () from /usr/lib/aarch64-linux-gnu/libc.so.6
#1  0x0000ffffff7d0a048 [PAC] in ?? () from /usr/lib/aarch64-linux-gnu/libc.so.6
#2  0x0000ffffff7d05a24 [PAC] in vfprintf () from /usr/lib/aarch64-linux-gnu/libc.so.6
#3  0x0000aaaaaab0011c [PAC] in __interceptor_fprintf ()
#4  0x0000aaaaaabbebd0 [PAC] in log_payload_error (error_description=0xfbfff5d00020 "%x%x%n\n")
    at sentinel_payload.c:35
#5  0x0000aaaaaabbf684 in parse_payload_file (fp=0xfd4ff6de0080) at sentinel_payload.c:99
#6  0x0000aaaaaabbfb70 in main (argc=2, argv=0xffffffffff258) at sentinel_payload.c:148
```

Exploitability Assessment:
Since this exploit allows attacks to print special format parameters, attackers could be able to reveal memory on the stack that they shouldn't be able to have access to. Using the `%n` special format parameter attackers could also potentially write to memory, especially if memory addresses for other variables are being stored on the stack.

4.2.6 Vulnerability 6

Field	Details
Vulnerability Name	`Stack buffer overflow in log_telecommand_header()`
CWE Classification	`CWE-121: Stack-based Buffer Overflow`
Target Component	`sentinel_network.c`

Vulnerable Location	`In log_telecommand_header on line 43`
	
	`sprintf(header_summary, "Command Type: %d, Length: %d, Magic: 0x%x, Data: %s", pkt→type, pkt→length, pkt->magic, (char *)pkt→data);`

Reproducing Input File	`crashing_inputs/network_crash_01.bin`

Triggering Input & Reproduction Command:

```
# Provide command to reproduce the crash
import struct

magic = 0×12345678
packet_type = 1 # Station status update type
payload_data =
b"00847793648074073640873804780374803740873084708456708734083780476508370487308470870827308270484074308470834708476976296397263972639726392637296392637925639726397263972639726392763926392763847793648074073640873804780374803740873084708456708734083780476508370487308470870827308270484074308470834708476976296397263972639726392637296392637925639726397263972639726392763926392763847793648074073640873804780374803740873084708456708734083780476508370487308470870827308270484074308470834708476976296397263972639726392637296392637925639726397263972639726392763926392763847793648074073640873804780374803740873084708456708734083780476508370487308470870827308270484074308470834708476976296397263972639726392637296392637925639726397263972639726392763926392763847793648074073640873804780374803740873084708456708734083780476508370487308470870827308270484074308470834708476976296397263972639726392637296392637925639726397263972639726392763926392763"
packet_len = len(payload_data)

# Pack header: 4-byte uint32 magic, 2-byte uint16 type, 2-byte uint16 length
packet_header = struct.pack("<IHH", magic, packet_type, packet_len)
```

```
with open("network_crash_01p5.bin", "wb") as f:
    f.write(packet_header + payload_data)
```

Root Cause Analysis:

In the function `log_telecommand_header()`, the `sprintf()` function on line 43 writes to `header_summary` which was only allocated 128 characters. If the combined data being written to `header_summary` is over 128 characters, a stack buffer overflow will occur where `sprintf` writes beyond the memory assigned for `header_summary`.

GDB / Sanitiser Evidence:

```
Address 0xfbfff5b002a0 is located in stack of thread T0 at offset 160 in frame
    #0 0xaaaaaabbe794  (/home/amber.guest/assignment2/sentinel_network+0x11e794) (BuildId: ab288cd5b63be08cd9cac230656b6f879e9c66ce)

  This frame has 1 object(s):
    [32, 160) 'header_summary' (line 36) <== Memory access at offset 160 overflows this variable
HINT: this may be a false positive if your program uses some custom stack unwind mechanism, swapcontext or vfork
      (longjmp and C++ exceptions *are* supported)
SUMMARY: AddressSanitizer: stack-buffer-overflow (/home/amber.guest/assignment2/sentinel_network+0x5ee44) (BuildId: ab288cd5b63be08cd9cac230656b6f879e9c66ce)
Shadow bytes around the buggy address:
  0xfbfff5b00000: f5 f5 f5 f5 f5 f5 f5 f5 f5 f5 f5 f5 f5 f5 f5 f5
  0xfbfff5b00080: f5 f5 f5 f5 f5 f5 f5 f5 f5 f5 f5 f5 f5 f5 f5 f5
  0xfbfff5b00100: f5 f5 f5 f5 f5 f5 f5 f5 f5 f5 f5 f5 f5 f5 f5 f5
  0xfbfff5b00180: f5 f5 f5 f5 f5 f5 f5 f5 f5 f5 f5 f5 f5 f5 f5 f5
  0xfbfff5b00200: f1 f1 f1 f1 00 00 00 00 00 00 00 00 00 00 00 00
=>0xfbfff5b00280: 00 00 00 00[f3]f3 f3 f3 f3 f3 f3 f3 f3 f3 f3 f3
  0xfbfff5b00300: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0xfbfff5b00380: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0xfbfff5b00400: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0xfbfff5b00480: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0xfbfff5b00500: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
Shadow byte legend (one shadow byte represents 8 application bytes):
  Addressable:           00
  Partially addressable: 01 02 03 04 05 06 07
  Heap left redzone:       fa
  Freed heap region:       fd
  Stack left redzone:      f1
  Stack mid redzone:       f2
  Stack right redzone:     f3
  Stack after return:      f5
  Stack use after scope:   f8
  Global redzone:          f9
  Global init order:       f6
  Poisoned by user:        f7
  Container overflow:      fc
  Array cookie:            ac
  Intra object redzone:    bb
  ASan internal:           fe
  Left alloca redzone:     ca
  Right alloca redzone:    cb
==2742==ABORTING
```

Exploitability Assessment:

This vulnerability is another stack buffer overflow. This can allow attacks that cause crashes, or that overwrite other variables on the stack. This vulnerability could also allow for overwriting return addresses, which could jump the program to any function.